

LAPORAN PRAKTIKUM PERTEMUAN 4
“Mengambil Data dari REST API (HTTP, Async, dan FutureBuilder)”



Disusun Oleh :
Adelia Rizky Cantika (20240801059)

Dosen Pengampu :
JEFRY SUNUPURWA ASRI , S.Kom., M.Kom

Mata Kuliah :
Pemrograman Mobile
CR002

FAKULTAS ILMU KOMPUTER
UNIVERSITAS ESA UNGGUL
2026

Langkah Praktikum

Bagian A: Mengenal Future dan FutureBuilder (tanpa internet)

```
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());
class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Praktikum 4',
      theme: ThemeData(colorSchemeSeed: Colors.blue, useMaterial3: true),
      home: const SalamPage(),
    );
  }
}
Future<String> ambilSalam() async {
  await Future.delayed(const Duration(seconds: 2));
  return 'Halo dari masa depan!';
}
class SalamPage extends StatefulWidget {
  const SalamPage({super.key});
  @override
  State<SalamPage> createState() => _SalamPageState();
}
class _SalamPageState extends State<SalamPage> {
  late Future<String> _future;
  @override
  void initState() {
    super.initState();
    _future = ambilSalam();
  }
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Demo Future')),
      body: Center(
        child: FutureBuilder<String>(
          future: _future,
          builder: (context, snapshot) {
            if (snapshot.connectionState == ConnectionState.waiting) {
              return const CircularProgressIndicator();
            }
            if (snapshot.hasError) {
              return Text('Error: ${snapshot.error}');
            }
          }
        )
      )
    );
  }
}
```

```

        return Text(snapshot.data!, style: const TextStyle(fontSize: 24));
      },
    ),
  ),
);
}
}

```

Tampilan:



Bagian C- E:

1. Model dan Fungsi Pengambil Data
2. Menampilkan Data dengan FutureBuilder
3. Halaman Detail

```
import 'dart:convert';
```

```
import 'package:flutter/material.dart';
```

```
import 'package:http/http.dart' as http;
```

```
void main() {
```

```
  runApp(const MyApp());
```

```
}
```

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      debugShowCheckedModeBanner: false,  
      title: 'Praktikum 4',  
      theme: ThemeData(  
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.blue),  
        useMaterial3: true,  
      ),  
      home: const PenggunaPage(),  
    );  
  }  
}
```

```
// =====  
// BAGIAN C  
// =====
```

```
class Pengguna {  
  final int id;  
  final String name;  
  final String email;  
  final String phone;  
  final String website;  
  
  Pengguna({
```

```
        required this.id,  
        required this.name,  
        required this.email,  
        required this.phone,  
        required this.website,  
    });
```

```
factory Pengguna.fromJson(Map<String, dynamic> json) {  
    return Pengguna(  
        id: json['id'],  
        name: json['name'],  
        email: json['email'],  
        phone: json['phone'],  
        website: json['website'],  
    );  
}  
}
```

```
Future<List<Pengguna>> ambilPengguna() async {  
    final uri = Uri.parse(  
        'https://jsonplaceholder.typicode.com/users',  
    );
```

```
    final response = await http  
        .get(uri)  
        .timeout(const Duration(seconds: 10));
```

```
    if (response.statusCode != 200) {  
        throw Exception(  
            'Gagal mengambil data: ${response.statusCode}',  
        );
```

```
}
```

```
final List<dynamic> jsonData = jsonDecode(response.body);
```

```
return jsonData
```

```
    .map((item) => Pengguna.fromJson(item))
```

```
    .toList();
```

```
}
```

```
// =====
```

```
// BAGIAN D
```

```
// =====
```

```
class PenggunaPage extends StatefulWidget {
```

```
    const PenggunaPage({super.key});
```

```
    @override
```

```
    State<PenggunaPage> createState() => _PenggunaPageState();
```

```
}
```

```
class _PenggunaPageState extends State<PenggunaPage> {
```

```
    late Future<List<Pengguna>> _future;
```

```
    @override
```

```
    void initState() {
```

```
        super.initState();
```

```
        _future = ambilPengguna();
```

```
}
```

```
void _muatUlang() {
```

```
    setState(() {
```

```
    _future = ambilPegguna();  
  });  
}
```

```
@override
```

```
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(  
      title: const Text('Daftar Pengguna'),  
      actions: [  
        IconButton(  
          onPressed: _muatUlang,  
          icon: const Icon(Icons.refresh),  
        ),  
      ],  
    ),  
    body: FutureBuilder<List<Pengguna>>(  
      future: _future,  
      builder: (context, snapshot) {  
        if (snapshot.connectionState ==  
            ConnectionState.waiting) {  
          return const Center(  
            child: CircularProgressIndicator(),  
          );  
        }  
  
        if (snapshot.hasError) {  
          return Center(  
            child: Padding(  
              padding: const EdgeInsets.all(16),  
              child: Column(  
                children: [
```

```

        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          const Icon(
            Icons.error,
            color: Colors.red,
            size: 48,
          ),
          const SizedBox(height: 8),
          Text(
            'Terjadi kesalahan: ${snapshot.error}',
            textAlign: TextAlign.center,
          ),
          const SizedBox(height: 12),
          ElevatedButton(
            onPressed: _muatUlang,
            child: const Text('Coba lagi'),
          ),
        ],
      ),
    ),
  );
}

```

```

final data = snapshot.data!;

```

```

return ListView.builder(
  itemCount: data.length,
  itemBuilder: (context, index) {
    final p = data[index];

    return ListTile(

```

```

        leading: CircleAvatar(
          child: Text(p.name[0]),
        ),
        title: Text(p.name),
        subtitle: Text(p.email),
        trailing: const Icon(
          Icons.chevron_right,
        ),
        onTap: () {
          Navigator.push(
            context,
            MaterialPageRoute(
              builder: (context) =>
                DetailPenggunaPage(
                  pengguna: p,
                ),
            ),
          );
        },
      );
    },
  ),
);
}
}

```

```
// =====
```

```
// BAGIAN E
```

```
// =====
```

```
class DetailPenggunaPage extends StatelessWidget {  
  final Pengguna pengguna;
```

```
  
  const DetailPenggunaPage({  
    super.key,  
    required this.pengguna,  
  });
```

```
  
  @override
```

```
  Widget build(BuildContext context) {
```

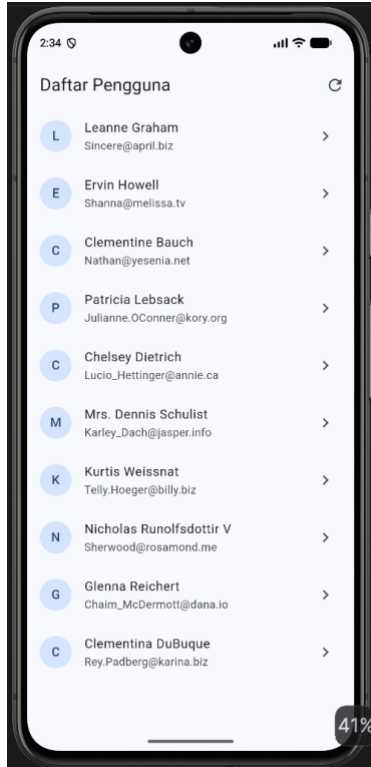
```
    return Scaffold(  
      appBar: AppBar(  
        title: Text(pengguna.name),  
      ),  
      body: ListView(  
        children: [  
          ListTile(  
            leading: const Icon(Icons.email),  
            title: Text(pengguna.email),  
          ),  
          ListTile(  
            leading: const Icon(Icons.phone),  
            title: Text(pengguna.phone),  
          ),  
          ListTile(  
            leading: const Icon(Icons.language),  
            title: Text(pengguna.website),  
          ),  
        ],  
      ),
```

);

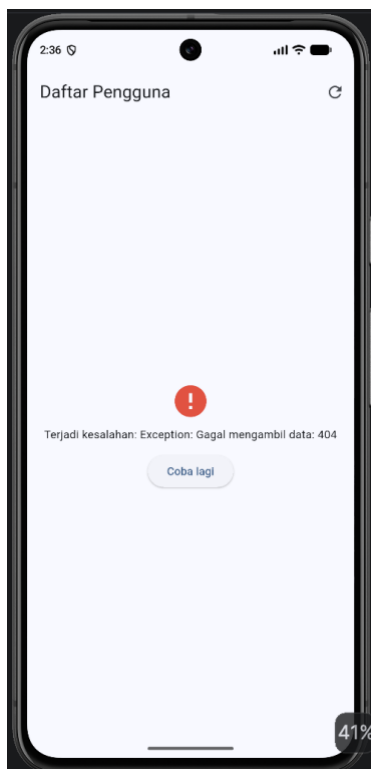
}

}

Tampilan:



Bagian F



Latihan 4:

```
import 'dart:convert';
```

```
import 'package:flutter/material.dart';  
import 'package:http/http.dart' as http;
```

```
void main() {  
  runApp(const MyApp());  
}
```

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});
```

```
  @override
```

```
  Widget build(BuildContext context) {  
    return MaterialApp(  
      debugShowCheckedModeBanner: false,  
      title: 'Praktikum 4',  
      theme: ThemeData(  
        colorScheme: ColorScheme.fromSeed(  
          seedColor: Colors.blue,  
        ),  
        useMaterial3: true,  
      ),  
      home: const PenggunaPage(),  
    );  
  }  
}
```

```
// =====  
// BAGIAN C  
// =====
```

```
class Pengguna {  
  final int id;  
  final String name;  
  final String username;  
  final String email;  
  final String phone;  
  final String website;  
  final String city;
```

```
  Pengguna({  
    required this.id,  
    required this.name,  
    required this.username,  
    required this.email,
```

```

        required this.phone,
        required this.website,
        required this.city,
    });

```

```

factory Pengguna.fromJson(Map<String, dynamic> json) {
    return Pengguna(
        id: json['id'],
        name: json['name'],
        username: json['username'],
        email: json['email'],
        phone: json['phone'],
        website: json['website'],
        city: json['address']['city'],
    );
}
}

```

```

Future<List<Pengguna>> ambilPengguna() async {
    final uri = Uri.parse(
        'https://jsonplaceholder.typicode.com/users',
    );

```

```

    final response = await http
        .get(uri)
        .timeout(const Duration(seconds: 10));

```

```

    if (response.statusCode != 200) {
        throw Exception(
            'Gagal mengambil data: ${response.statusCode}',
        );
    }

```

```

    final List<dynamic> jsonData = jsonDecode(response.body);

```

```

    return jsonData
        .map((item) => Pengguna.fromJson(item))
        .toList();
}

```

```

// =====
// BAGIAN D + LATIHAN 2, 3, DAN 4
// =====

```

```

class PenggunaPage extends StatefulWidget {
    const PenggunaPage({super.key});

```

```

@override
State<PenggunaPage> createState() => _PenggunaPageState();
}

class _PenggunaPageState extends State<PenggunaPage> {
  late Future<List<Pengguna>>> _future;

  @override
  void initState() {
    super.initState();
    _future = ambilPengguna();
  }

  void _muatUlang() {
    setState(() {
      _future = ambilPengguna();
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: FutureBuilder<List<Pengguna>>>(
          future: _future,
          builder: (context, snapshot) {
            // Latihan Mandiri 4:
            // Menampilkan jumlah pengguna setelah data berhasil dimuat
            if (snapshot.hasData) {
              return Text(
                'Daftar Pengguna (${snapshot.data!.length})',
              );
            }

            return const Text('Daftar Pengguna');
          },
        ),
        actions: [
          IconButton(
            onPressed: _muatUlang,
            icon: const Icon(Icons.refresh),
          ),
        ],
      ),
      body: FutureBuilder<List<Pengguna>>>(
        future: _future,
        builder: (context, snapshot) {

```

```

// Loading
if (snapshot.connectionState ==
    ConnectionState.waiting) {
    return const Center(
        child: CircularProgressIndicator(),
    );
}

// Error
if (snapshot.hasError) {
    return Center(
        child: Padding(
            padding: const EdgeInsets.all(16),
            child: Column(
                mainAxisAlignment: MainAxisAlignment.min,
                children: [
                    const Icon(
                        Icons.error,
                        color: Colors.red,
                        size: 48,
                    ),
                    const SizedBox(height: 8),
                    Text(
                        'Terjadi kesalahan: ${snapshot.error}',
                        textAlign: TextAlign.center,
                    ),
                    const SizedBox(height: 12),
                    ElevatedButton(
                        onPressed: _muatUlang,
                        child: const Text('Coba lagi'),
                    ),
                ],
            ),
        ),
    );
}

```

```

final data = snapshot.data!;

```

```

// Latihan Mandiri 3:

```

```

// Jika data kosong

```

```

if (data.isEmpty) {
    return const Center(
        child: Text('Tidak ada data'),
    );
}

```

```

// Latihan Mandiri 2:
// Pull-to-refresh
return RefreshIndicator(
  onRefresh: () async {
    setState(() {
      _future = ambilPengguna();
    });

    await _future;
  },
  child: ListView.builder(
    itemCount: data.length,
    itemBuilder: (context, index) {
      final p = data[index];

      return ListTile(
        leading: CircleAvatar(
          child: Text(p.name[0]),
        ),
        title: Text(p.name),
        subtitle: Text(p.email),
        trailing: const Icon(
          Icons.chevron_right,
        ),
        onTap: () {
          Navigator.push(
            context,
            MaterialPageRoute(
              builder: (context) =>
                DetailPenggunaPage(
                  pengguna: p,
                ),
            ),
          );
        },
      );
    },
  ),
);
}
}

// =====
// BAGIAN E + LATIHAN 1

```

```
// =====
```

```
class DetailPenggunaPage extends StatelessWidget {  
  final Pengguna pengguna;
```

```
  
  const DetailPenggunaPage({  
    super.key,  
    required this.pengguna,  
  });
```

```
  @override
```

```
  Widget build(BuildContext context) {
```

```
    return Scaffold(  
      appBar: AppBar(  
        title: Text(pengguna.name),  
      ),  
      body: ListView(  
        children: [  
          // Latihan Mandiri 1: Username  
          ListTile(  
            leading: const Icon(Icons.person),  
            title: Text(pengguna.username),  
          ),  
          // Bagian E: Email  
          ListTile(  
            leading: const Icon(Icons.email),  
            title: Text(pengguna.email),  
          ),  
          // Bagian E: Telepon  
          ListTile(  
            leading: const Icon(Icons.phone),  
            title: Text(pengguna.phone),  
          ),  
          // Bagian E: Website  
          ListTile(  
            leading: const Icon(Icons.language),  
            title: Text(pengguna.website),  
          ),  
          // Latihan Mandiri 1: Kota  
          ListTile(  
            leading: const Icon(Icons.location_city),  
            title: Text(pengguna.city),  
          ),  
        ],  
      ),  
    );  
  }
```

```

    ],
  ),
);
}
}

```

Tampilan:



Tugas 6:

```
import 'dart:convert';
```

```
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
```

```
void main() {
  runApp(const MyApp());
}
```

```
// =====
// APLIKASI
// =====
```

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});
```

```
  @override
  Widget build(BuildContext context) {
```

```
return MaterialApp(
  debugShowCheckedModeBanner: false,
  title: 'Daftar Postingan',
  theme: ThemeData(
    colorScheme: ColorScheme.fromSeed(
      seedColor: Colors.blue,
    ),
    useMaterial3: true,
  ),
  home: const PostPage(),
);
}
}
```

```
// =====
// MODEL POST
// =====
```

```
class Post {
  final int userId;
  final int id;
  final String title;
  final String body;

  Post({
    required this.userId,
    required this.id,
    required this.title,
    required this.body,
  });

  factory Post.fromJson(Map<String, dynamic> json) {
    return Post(
      userId: json['userId'],
      id: json['id'],
      title: json['title'],
      body: json['body'],
    );
  }
}
```

```
// =====
// MODEL KOMENTAR
// =====
```

```
class Komentar {
  final int postId;
```

```
final int id;
final String name;
final String email;
final String body;
```

```
Komentar({
    required this.postId,
    required this.id,
    required this.name,
    required this.email,
    required this.body,
});
```

```
factory Komentar.fromJson(Map<String, dynamic> json) {
    return Komentar(
        postId: json['postId'],
        id: json['id'],
        name: json['name'],
        email: json['email'],
        body: json['body'],
    );
}
}
```

```
// =====
// FUNGSI MENGAMBIL DATA POSTINGAN
// =====
```

```
Future<List<Post>> ambilPostingan() async {
    final uri = Uri.parse(
        'https://jsonplaceholder.typicode.com/posts',
    );

    final response = await http
        .get(uri)
        .timeout(const Duration(seconds: 10));

    if (response.statusCode != 200) {
        throw Exception(
            'Gagal mengambil postingan: ${response.statusCode}',
        );
    }

    final List<dynamic> jsonData = jsonDecode(response.body);

    return jsonData
        .map((item) => Post.fromJson(item))
}
```

```

        .toList();
    }

// =====
// FUNGSI MENGAMBIL KOMENTAR
// =====

Future<List<Komentar>> ambilKomentar(int postId) async {
    final uri = Uri.parse(
        'https://jsonplaceholder.typicode.com/posts/$postId/comments',
    );

    final response = await http
        .get(uri)
        .timeout(const Duration(seconds: 10));

    if (response.statusCode != 200) {
        throw Exception(
            'Gagal mengambil komentar: ${response.statusCode}',
        );
    }

    final List<dynamic> jsonData = jsonDecode(response.body);

    return jsonData
        .map((item) => Komentar.fromJson(item))
        .toList();
}

// =====
// HALAMAN DAFTAR POSTINGAN
// =====

class PostPage extends StatefulWidget {
    const PostPage({super.key});

    @override
    State<PostPage> createState() => _PostPageState();
}

class _PostPageState extends State<PostPage> {
    late Future<List<Post>> _future;

    @override
    void initState() {
        super.initState();
        _future = ambilPostingan();
    }
}

```

```
}
```

```
void _muatUlang() {  
  setState(() {  
    _future = ambilPostingan();  
  });  
}
```

```
@override
```

```
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(  
      title: const Text('Daftar Postingan'),  
      actions: [  
        IconButton(  
          onPressed: _muatUlang,  
          icon: const Icon(Icons.refresh),  
        ),  
      ],  
    ),  
    body: FutureBuilder<List<Post>>(  
      future: _future,  
      builder: (context, snapshot) {  
        // LOADING  
        if (snapshot.connectionState ==  
            ConnectionState.waiting) {  
          return const Center(  
            child: CircularProgressIndicator(),  
          );  
        }  
  
        // ERROR  
        if (snapshot.hasError) {  
          return Center(  
            child: Padding(  
              padding: const EdgeInsets.all(20),  
              child: Column(  
                mainAxisAlignment: MainAxisAlignment.min,  
                children: [  
                  const Icon(  
                    Icons.error_outline,  
                    color: Colors.red,  
                    size: 50,  
                  ),  
                  const SizedBox(height: 12),  
                  Text(  
                    'Terjadi kesalahan:\n${snapshot.error}',
```

```

        textAlign: TextAlign.center,
      ),
      const SizedBox(height: 16),
      ElevatedButton(
        onPressed: _muatUlang,
        child: const Text('Coba lagi'),
      ),
    ],
  ),
);
}

```

// DATA

```
final data = snapshot.data ?? [];
```

```

if (data.isEmpty) {
  return const Center(
    child: Text('Tidak ada data'),
  );
}

```

// DAFTAR POSTINGAN

```

return ListView.builder(
  itemCount: data.length,
  itemBuilder: (context, index) {
    final post = data[index];

    return ListTile(
      leading: CircleAvatar(
        child: Text('${post.id}'),
      ),
      title: Text(
        post.title,
        maxLines: 2,
        overflow: TextOverflow.ellipsis,
      ),
      subtitle: Text(
        post.body,
        maxLines: 2,
        overflow: TextOverflow.ellipsis,
      ),
      trailing: const Icon(
        Icons.chevron_right,
      ),
      onTap: () {
        Navigator.push(

```

```

        context,
        MaterialPageRoute(
          builder: (context) =>
            DetailPostPage(post: post),
        ),
      );
    },
  );
},
);
},
),
);
}
}

// =====
// HALAMAN DETAIL POSTINGAN
// =====

```

```

class DetailPostPage extends StatefulWidget {
  final Post post;

  const DetailPostPage({
    super.key,
    required this.post,
  });

  @override
  State<DetailPostPage> createState() =>
    _DetailPostPageState();
}

class _DetailPostPageState extends State<DetailPostPage> {
  late Future<List<Komentar>>> _futureKomentar;

  @override
  void initState() {
    super.initState();

    _futureKomentar =
      ambilKomentar(widget.post.id);
  }

  void _muatUlangKomentar() {
    setState(() {
      _futureKomentar =

```

```
        ambilKomentar(widget.post.id);
    });
}
```

```
@override
Widget build(BuildContext context) {
    return Scaffold(
        appBar: AppBar(
            title: const Text('Detail Postingan'),
        ),
        body: ListView(
            padding: const EdgeInsets.all(16),
            children: [
                // JUDUL POSTINGAN
                Text(
                    widget.post.title,
                    style: const TextStyle(
                        fontSize: 22,
                        fontWeight: FontWeight.bold,
                    ),
                ),
                const SizedBox(height: 16),

                // ISI LENGKAP POSTINGAN
                Text(
                    widget.post.body,
                    style: const TextStyle(
                        fontSize: 16,
                        height: 1.5,
                    ),
                ),
                const SizedBox(height: 24),

                const Divider(),

                const SizedBox(height: 12),

                // JUDUL KOMENTAR
                const Text(
                    'Komentar',
                    style: TextStyle(
                        fontSize: 20,
                        fontWeight: FontWeight.bold,
                    ),
                ),
            ],
        ),
    );
}
```

```

const SizedBox(height: 12),

// FUTUREBUILDER KOMENTAR
FutureBuilder<List<Komentar>>(  
  future: _futureKomentar,  
  builder: (context, snapshot) {  
    // LOADING KOMENTAR  
    if (snapshot.connectionState ==  
      ConnectionState.waiting) {  
      return const Padding(  
        padding: EdgeInsets.all(20),  
        child: Center(  
          child: CircularProgressIndicator(),  
        ),  
      );  
    }  
  
    // ERROR KOMENTAR  
    if (snapshot.hasError) {  
      return Padding(  
        padding: const EdgeInsets.all(16),  
        child: Column(  
          children: [  
            const Icon(  
              Icons.error_outline,  
              color: Colors.red,  
              size: 45,  
            ),  
            const SizedBox(height: 8),  
            Text(  
              'Gagal memuat komentar:\n'  
                '${snapshot.error}',  
              textAlign: TextAlign.center,  
            ),  
            const SizedBox(height: 12),  
            ElevatedButton(  
              onPressed: _muatUlangKomentar,  
              child: const Text('Coba lagi'),  
            ),  
          ],  
        ),  
      );  
    }  
  
    // DATA KOMENTAR  
    final komentar =

```

```

        snapshot.data ?? []];

    if (komentar.isEmpty) {
      return const Center(
        child: Text('Tidak ada komentar'),
      );
    }

    // DAFTAR KOMENTAR
    return Column(
      children: komentar.map((item) {
        return Card(
          margin: const EdgeInsets.only(
            bottom: 12,
          ),
          child: Padding(
            padding: const EdgeInsets.all(12),
            child: Column(
              crossAxisAlignment:
                CrossAxisAlignment.start,
              children: [
                Text(
                  item.name,
                  style: const TextStyle(
                    fontWeight: FontWeight.bold,
                  ),
                ),
                const SizedBox(height: 4),
                Text(
                  item.email,
                  style: const TextStyle(
                    fontSize: 13,
                  ),
                ),
                const SizedBox(height: 8),
                Text(item.body),
              ],
            ),
          ),
        ).toList(),
    );
  },
),
],
),
);

```

Tampilan:



8. Pertanyaan Refleksi

1. Apa yang terjadi bila Future dibuat di dalam build() dan bukan di initState()? Mengapa?

Data bisa dimuat berulang kali setiap halaman diperbarui. Makanya, Future lebih baik dibuat di initState() supaya data tidak terus-menerus dimuat ulang.

2. Apa perbedaan snapshot.hasError dengan memeriksa response.statusCode? Mengapa keduanya diperlukan?

snapshot.hasError digunakan untuk mengecek apakah ada kesalahan saat mengambil data, sedangkan response.statusCode untuk mengecek apakah permintaan ke server berhasil atau tidak. Keduanya diperlukan supaya kita tahu penyebab data gagal dimuat.

3. Mengapa data JSON sebaiknya diubah menjadi class model, bukan dipakai langsung sebagai Map?

Supaya data lebih rapi dan mudah dipahami. Dengan class model, kita juga lebih mudah mengambil data yang dibutuhkan dan mengurangi kesalahan saat menulis kode.

4. Mengapa UI wajib menyediakan status loading dan error, bukan hanya status data?

Supaya pengguna tahu kalau data sedang dimuat atau sedang terjadi masalah. Jadi, pengguna tidak bingung saat data belum muncul atau gagal ditampilkan.